Vision-Lab-1

Start Assignment

- Due Thursday by 11:59pm
- Points 20
- Submitting a file upload

Welcome to our first Vision Lab! This week, we will learn how to install OpenCV with Python and develop a camera app featuring practical functions for everyday use.

PART 1

For the requirements of our labs, we will need **Python3 (https://www.python.org)** , **PIP (https://pypi.org/project/pip/)** and **OpenCV (https://opencv.org)** . Additional packages will be necessary and will be installed while we are developing our course.

Install Python3

Open the terminal in your ubuntu system. The following command will download the Python directly onto your machine.

$ sudo apt-get install python3

Install pip

Pip is a package installer for Python which you can easily manage the installation of Python packages.

$ sudo apt-get install python3-pip

Install OpenCV library with pip

$ pip3 install opencv-python

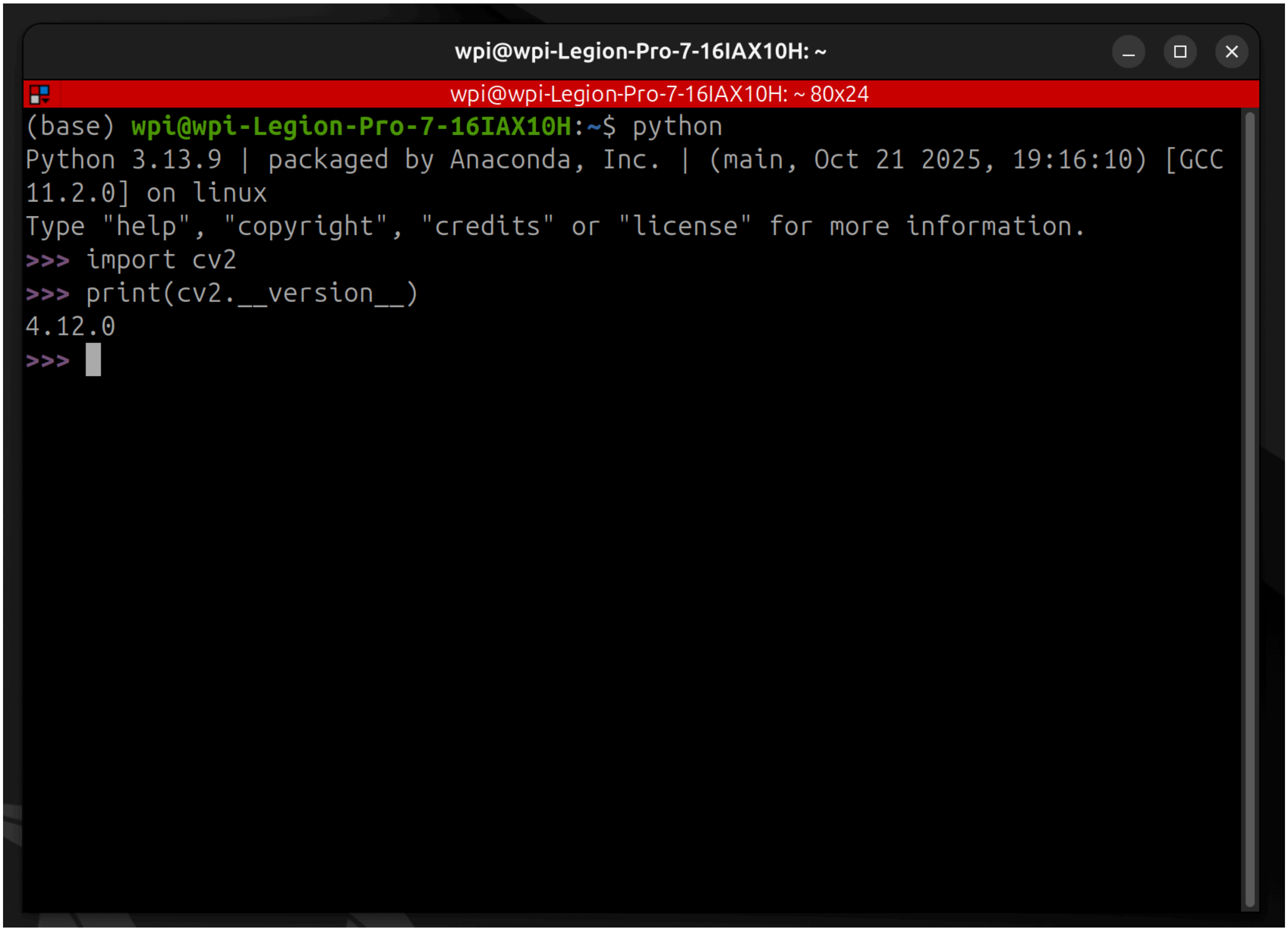
Verify Python3 and OpenCV versions

After the installation of OpenCV, type $*python3* and verify that you are using Python version >= 3.13.x and OpenCV version >= 4.12.x by typing the following lines in the python command line.

>>> import cv2

>>> print(cv2.__version__)

*Please send me a screenshot of your terminal indicating your Python3 and OpenCV versions (**1 point**)*



There are also **other ways to install OpenCV (https://docs.opencv.org/4.x/d2/de6/tutorial_py_setup_in_ubuntu.html)** such as installation from pre-build binaries available in Ubuntu repositories or compile OpenCV from source using *cmake*.

https://docs.opencv.org/4.x/d2/de6/tutorial_py_setup_in_ubuntu.html (https://docs.opencv.org/4.x/d2/de6/tutorial_py_setup_in_ubuntu.html)

To avoid any future version conflicts or confusions, I would encourage you to use pip.

PART 2

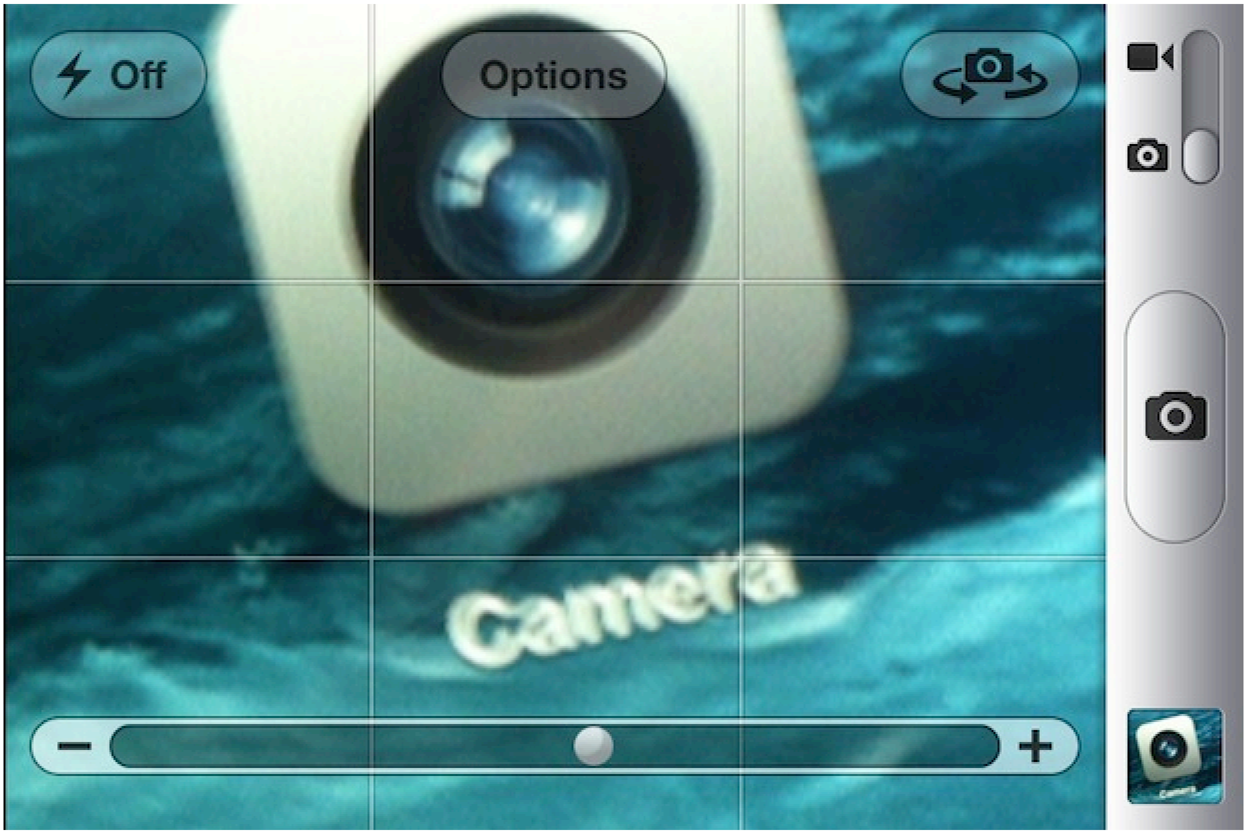
Follow the **GUI Features tutorials (https://docs.opencv.org/4.x/dc/d4d/tutorial_py_table_of_contents_gui.html)** below to learn how to load images, videos, draw shapes and text plus add trackbars and controls.

https://docs.opencv.org/4.x/dc/d4d/tutorial_py_table_of_contents_gui.html (https://docs.opencv.org/4.x/dc/d4d/tutorial_py_table_of_contents_gui.html)

Next, develop a "**Photo Booth (https://support.apple.com/guide/photo-booth/take-a-photo-or-record-a-video-pbhlp3714a9d/mac)** " or "**Camera (https://apps.microsoft.com/store/detail/windows-camera/9WZDNCRFJBBG)** " application using the fundamental features outlined below. Utilize your webcam to capture and store images/video on your computer. To interact with the application, use the **'c'** key to save an image, **'v'** to start/stop recording, and the **'esc'** key to exit (***3 points***).

You also have the option to incorporate buttons instead of relying on keyboard input. (*optional*). Make sure that the saved images and videos feature a Date/Time stamp in the bottom right corner, mimicking the appearance of traditional cameras (***1 point***).

Implement a **trackbar (https://osxdaily.com/2012/04/18/zoom-camera-iphone/)** functionality that enables zooming in and out during live streaming (***2 points***).



PART 3

Follow the below two tutorials and learn how to access pixels values, set a Region of Interest (ROI) and perform several arithmetic operations on images.

https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html
(https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html)

https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html
(https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html)

Then, in your PhotoBooth/Camera app add the following features:

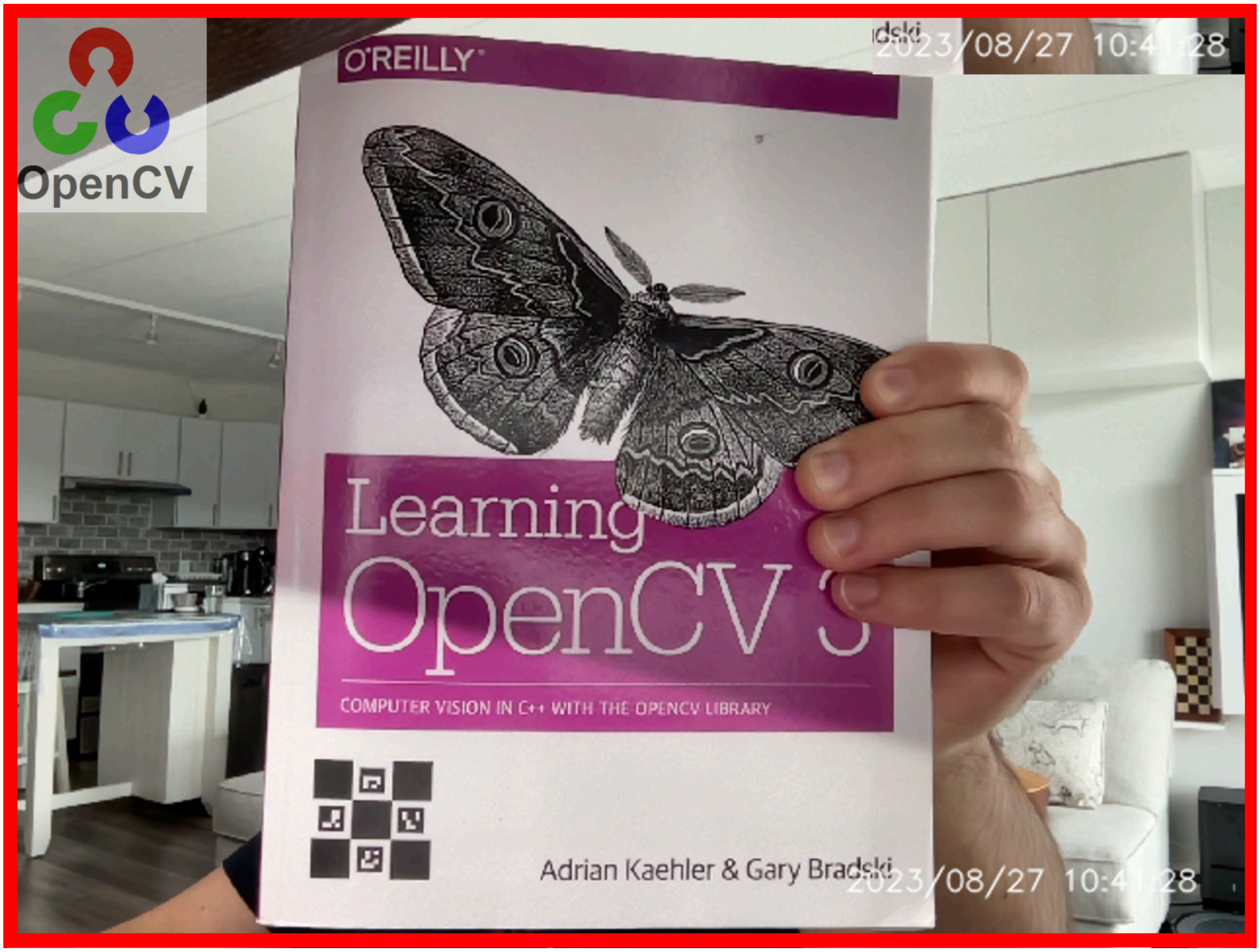
a) When capturing an photo by pressing **'c'**, make the screen flash (change all the pixels to white) (***1 point***)

b) Select the Date/Time stamp area added in the previous lab as a ROI and copy it at the top right corner (***1 point***)

c) Add a red constant border in your image/camera-stream (***1 point***)

d) Blend the OpenCV logo below in your image/camera-stream at the top left corner (***1 point***)

OpenCV



PART 4

Follow the below four tutorials and learn how to convert images in other color-spaces, perform geometric transformations, thresholding and apply filters to images.

https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
(https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html)

https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html (https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html)

https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html (https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html)

https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html (https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html)

Then, in your PhotoBooth/Camera app add the following features:

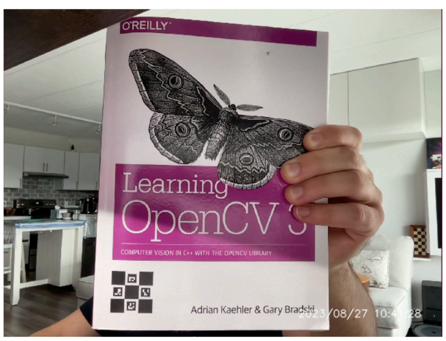
a) By pressing **'e'** extract the pink color (or any color you prefer) using the bitwise-AND mask on the original image/camera-stream. (***1 point***)

b) By pressing **'r'** rotate the image/camera-stream by 10 degrees. (***1 point***)

c) By pressing **'t'** threshold the image/camera-stream (optional thresholds) (***1 point***)

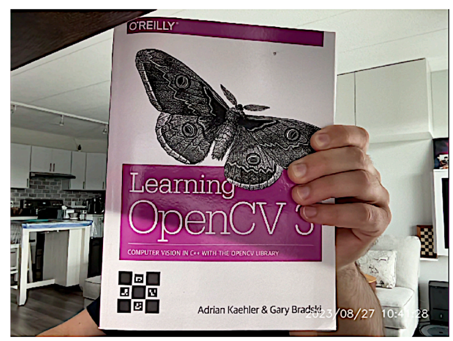
d) By pressing **'b'** and using a trackbar apply Gaussian blurring in your image/camera-stream. Adjust the trackbar to set the sigma values (X and Y) from (5-30). (2 ***points***)

e) By pressing **'s'** extract the sharpened image (***1 point***)

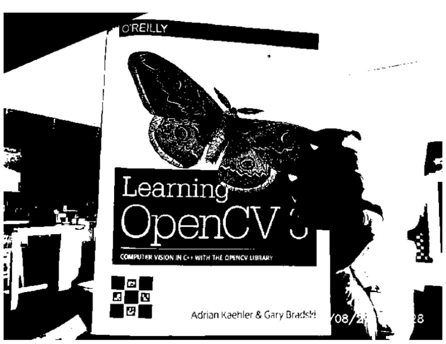


Original

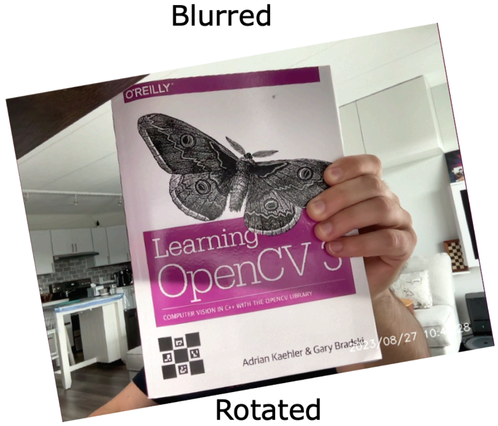
Blurred



Sharpened



Thresholding



Rotated

PART 5

Please submit your Python3 code file with explanatory comments, a document, named **Vision-Lab1-Report.pdf**, with your outcomes as were illustrated in this lab, and a **screen-video recording** of your application while performing the key actions requested (***3 points***).

You are highly encouraged to let your creativity flow and introduce additional features, keeping in mind the general concept outlined for your application.

Week 1 Discussion (https://canvas.wpi.edu/courses/87347/discussion_topics/394306)

Good luck and have fun